

创意实践

播客生成

 Copy page

场景介绍

PodLM 是中文版播客开源项目，可以将任意 URL 内容转换成播客。

使用场景

Google 的 NotebookLM 项目通过将文本转换为对话式的音频从而达到模拟播客的效果。其基本任务流程是通过大模型读取理解文本内容并根据提示词、人设等生成对话式的文本，并通过音频合成方案生成对话音频。除了生成播客，该方案可以在诸多其它场景进行复用，例如：

- 内容摘要：可以将新闻、资讯文章、论文等内容转换成播客音频，以便在上班通勤等不便持续阅读的场景通过播客音频的方式快速了解知识资讯的内容；
- 会议回顾：可以通过会议全程记录提炼重点摘要，并生成音频，让未能参会的同事快速身临其境的回顾会议内容，重点内容不错过；
- 场景教育：可以根据课程材料和教学场景，生成内容音频，增加教学的互动性和趣味性，提升教学效果；
- 育儿成长：可以将儿童故事等生成生动有趣的对话音频，作为儿童睡前故事播放，让爸爸妈妈不在的时候也有故事相伴；但是 NotebookLM 项目目前对中文支持不佳，且在音频风格等方面未开放更多选项。得益于开源社区项目 PodLM，现在可以通过智谱 BigModel 大模型配合音频合成服务，生成具有多变风格的中文版播客。

方案

PodLM 项目已在 GitHub 开源，你可以通过多种方式将其部署到本地，并根据官方文档启动项目。官方文档：<https://github.com/lihuithe/podlm-public>

1. TTS 方案

由于 PodLM 没有提供文本到语音（Text-to-Speech, TTS）的实现方案，我们可以自行采取合适的方案实现，并支持个性化。

TTS 实现参考：

```
import logging
import requests

from fastapi import FastAPI, HTTPException, BackgroundTasks
from fastapi.responses import FileResponse
import httpx
import os
import tempfile
from urllib.parse import urlencode
from urllib.parse import quote_plus

# Set up the logger
logging.basicConfig(level=logging.INFO, format="%(asctime)s - %(levelname)s - %(message)s")
logger = logging.getLogger(__name__)

API_KEY = "your baidu api key"
SECRET_KEY = "your baidu secret key"

# 发音人选择, 基础音库: 0为度小美, 1为度小宇, 3为度逍遥, 4为度丫丫,
# 精品音库: 5为度小娇, 103为度米朵, 106为度博文, 110为度小童, 111为度小萌, 默认为度小美
PER = 0
PER_Guest = 1

# 语速, 取值0-15, 默认为5中语速
SPD = 5

# 音调, 取值0-15, 默认为5中音调
PIT = 5

# 音量, 取值0-9, 默认为5中音量
VOL = 5

# 下载的文件格式, 3: mp3(default) 4: pcm-16k 5: pcm-8k 6: wav
AUE = 6
CUID = "123456PYTHON"

app = FastAPI()

def get_access_token():
```



def get_access_token() -> str: 生成鉴权签名 (Access Token)

:return: access_token, 或是None(如果错误)

"""

```
>
url = "https://aip.baidubce.com/oauth/2.0/token"
params = {"grant_type": "client_credentials", "client_id": API_KEY, "client_secret": API_SECRET_KEY}
return str(requests.post(url, params=params).json().get("access_token"))
```

API_URL = 'http://tsn.baidu.com/text2audio'

@app.get("/tts")

```
async def text_to_speech(text: str, background_tasks: BackgroundTasks, voice: str):
    temp_wav_file = tempfile.NamedTemporaryFile(suffix=".wav", delete=False)
    logger.info(f"Created temp WAV file: {temp_wav_file.name}")
    if voice == "host":
        params = {'tok': get_access_token(), 'tex': quote_plus(text), 'per': PER_HOST}
    else:
        params = {'tok': get_access_token(), 'tex': quote_plus(text), 'per': PER_COOPERATIVE}
    headers = {
        'Content-Type': 'application/json',
        'Accept': 'audio/wav'
    }
    data = urlencode(params)
    async with httpx.AsyncClient() as client:
        try:
            logger.info("Sending request ...")
            response = await client.post(API_URL, headers=headers, data=data.encode())
            response.raise_for_status() # Raise for HTTP errors
            # Log the content type of the response
            content_type = response.headers.get('Content-Type', 'unknown')
            logger.info(f"Response Content-Type: {content_type}")
            # Save the response content as an WAV file
            with open(temp_wav_file.name, "wb") as audio_file:
                audio_file.write(response.content)
            logger.info(f"Audio written to temp WAV file: {temp_wav_file.name}")
            # Use FileResponse to send the WAV file
            file_response = FileResponse(temp_wav_file.name, media_type="audio/wav")
            logger.info("Returning the WAV audio file.")
            # Add a background task to delete the files after response is sent
            background_tasks.add_task(os.remove, temp_wav_file.name)
```

```

        return file_response
    except httpx.TimeoutException:
        logger.error("Request timed out. Consider increasing the timeout limit")
        raise HTTPException(status_code=504, detail="Gateway Timeout: OpenAI /")
    except Exception as e:
        logger.error(f"Error occurred: {str(e)}")
        raise HTTPException(status_code=500, detail=str(e))

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=5012)    # 在 PodLM 项目的后端服务 server.

```

2. PodLM 部署

将 PodLM 项目部署到本地后，首先需要将 LLM 服务接口配置为智谱 BigModel 大模型，并接入 TTS 服务。

2.1 配置 GLM

- 在项目文件夹中找到后端服务实现 [server.py](#) ([server_pro.py](#)) ;
- 将源文件中所有的 `api_url` 、 `api_key` 、 `model` 配置为智谱 BigModel 大模型，例如：



```
def generate_podcast_title(content):
    def llm_request():
        api_url = "https://open.bigmodel.cn/api/paas/v4"    # 修改请求 URL
        api_key = "your zhipu api key"    # 添加 api key
        model = 'glm-4.7'    # 要调用的 GLM
        headers = {
            'Content-Type': 'application/json',
            'Authorization': f'Bearer {api_key}'
        }
        data = {
            'model': model,
            'messages': [
                {'role': 'system', 'content': '你是一个播客标题生成器，请根据给定的内容生成播客标题'},
                {'role': 'user', 'content': f"请为以下内容生成一个播客标题:\n{content}"}
            ]
        }
        response = requests.post(api_url, headers=headers, json=data)
        response.raise_for_status()
        result = response.json()
        if 'choices' in result and len(result['choices']) > 0:
            return result['choices'][0]['message']['content'].strip()
        else:
            raise ValueError('API返回的数据格式不正确')
```

2.2 接入 TTS

- 配置 [server.py](#) (server_[pro.py](#)) 中的音频服务接口。



```
def tts_request(text, anchor_type):
    if anchor_type == "主播Carol":
        voice = "host"
        url = f"http://127.0.0.1:5012/tts?text={text}&voice={voice}"    # 与 tts.p
    else:
        voice = "guest"
        url = f"http://127.0.0.1:5012/tts?text={text}&voice={voice}"    # 与 tts.p
```

2.3 修改提示词

- PodLM 项目的请求 messages 内容与 GLM 的 JSON 输出不完全匹配，需要简单修改，参考如下：

```
data = {
    'model': model,
    # 修改 messages 的内容
    'messages': [
        {'role': 'system', 'content': '你是一个播客对话内容生成器,你需要将我给你的内容'}
        {'role': 'user', 'content': f"请将以下内容转换成播客对话,对话内容content加身"}
    ]
}

log("正在发送第一次请求到 LLM API")
response = requests.post(api_url, headers=headers, json=data)
if response.status_code == 200:
    log("成功接收第一次 LLM API 响应")
    result = response.json()
    if 'choices' in result and len(result['choices']) > 0:
        content = result['choices'][0]['message']['content']
        log(f"API 返回的原始内容: {content}")
        content = content.replace('```json', '').replace('```', '').strip()
        try:
            dialogue = json.loads(content)
            all_content.extend(dialogue)
            log(f"成功解析第一次对话内容, 共 {len(dialogue)} 条对话")
        except json.JSONDecodeError as e:
            log(f"JSON 解析错误: {str(e)}")
            log("尝试修复 JSON 格式")
            fixed_content = content.replace('"', '').replace('\n', '\\n')
            try:
                dialogue = json.loads(fixed_content)
                all_content.extend(dialogue)
                log(f"修复后成功解析对话内容, 共 {len(dialogue)} 条对话")
            except json.JSONDecodeError as e:
                log(f"修复后仍然无法解析 JSON: {str(e)}")
                return []
    else:
        log(f"第一次生成对话内容失败, 状态码: {response.status_code}")
        return []
```

3. 启动项目

在不同的终端命令行分别运行 [tts.py](#)、[api.py](#)、[server.py](#) 即可启动服务，访问 <http://127.0.0.1:8811> 在输入框填入 URL 即可开始播客生成。



方案亮点

- 零门槛高效创作：让播客生产“从几天到几分钟”
- 内容形态多元创新：从“单一叙事”到“跨界融合”
- 沉浸式听觉体验：从“单一声线”到“多角色场景化”

[汉语新解](#)

[编辑视频](#)